

Manual de Uso — Cluster DGX / Slurm

Este documento estabelece as diretrizes obrigatórias para uso do cluster DGX e apresenta um mini-guia prático de submissão de jobs via Slurm.



Diretrizes Gerais de Uso

1. Armazenamento

⚠ **Não utilize a `home` para armazenamento de dados ou caches de execução.**

Cada usuário possui um diretório dedicado em:

```
/raid/<nome_do_usuario>
```

Todos os dados, modelos, checkpoints, datasets e imagens de container devem ser armazenados exclusivamente nesse diretório.

Boas práticas:

- Use a `home` apenas para arquivos leves (scripts, configs, chaves SSH).
- Configure variáveis de cache (HuggingFace, pip, torch, etc.) para `/raid/$USER`.
- Mantenha organização por projeto.

2. Execução de Jobs

⚠ **Todas as tarefas devem ser submetidas exclusivamente via Slurm.**

A execução direta fora do Slurm:

- Compromete a estabilidade do sistema
- Interfere no agendamento de recursos
- Pode causar bloqueio automático de acesso
- Afeta outros usuários

Não é permitido executar processos que utilizem GPU diretamente via `python` , `torchrun` , `docker` , etc., fora do Slurm.

3. Suporte e Comunicação

Para comunicação oficial, dúvidas e avisos:

link enviado no email



Mini-Guia de Uso do Slurm

Partições Disponíveis

PARTITION	NODE
h100n2	dgx-H100-02
h100n3	dgx-H100-03
b200n1	dgx-B200-1

Utilize sempre a partição correspondente à máquina onde seus dados estão armazenados.

Verificar Estado dos Nodes

```
sinfo
```

Estados comuns:

- `idle` → disponível
- `mix` → parcialmente ocupado
- `alloc` → totalmente ocupado (não apenas GPU)

Se estiver em `idle` ou `mix` , o node está apto a receber novos jobs.



Job Simples em Bash

Submissão a partir de um script:

```
sbatch job.sh
```

Monitoramento:

```
squeue -u $USER
```



Job com Python

Arquivo: job_python.sh

```
#!/bin/bash
#SBATCH --job-name=python
#SBATCH --partition=h100n3
#SBATCH --gres=gpu:1
#SBATCH --output=python_%j.log
```

```
module load python/3.10
python3 meu_script.py
```

Submeter:

```
sbatch job_python.sh
```



Job com Container (Apptainer)

Arquivo: job_container.sh

```
#!/bin/bash
#SBATCH --job-name=container
#SBATCH --partition=h100n3
#SBATCH --gres=gpu:1
#SBATCH --output=container_%j.log

# Caminho da imagem .sif (armazenar no /raid)
IMAGE=/raid/$USER/nvcr.io_nvidia_pytorch_24.03-py3.sif

# Execução com GPU e bind do diretório do usuário
apptainer exec --nv \
  -B /raid/$USER:/workspace \
  $IMAGE \
  python3 /workspace/treino.py
```

Criar ou Baixar Imagem de Container

```
apptainer pull nvcr.io/nvidia/pytorch:24.03-py3
```

Recomenda-se armazenar a imagem .sif dentro de:

```
/raid/$USER/containers/
```



Comandos Úteis

Ver uso de GPU:

```
nvidia-smi
```

Cancelar job:

```
scancel <job_id>
```

Detalhes de um job:

```
scontrol show job <job_id>
```

Checkpoints e Interrupções Programadas

⚠ O uso de checkpoint é fortemente recomendado e, em alguns casos, obrigatório.

O cluster pode passar por **manutenções programadas ou emergenciais**, que exigem:

- parada temporária da fila Slurm
- cancelamento de jobs em execução
- reinício de nós ou serviços críticos

Jobs **sem checkpoint** podem perder **100% do progresso** nesses cenários.

Diretrizes sobre checkpoints

- Sempre que possível, implemente **checkpoint periódico** no código.
- Prefira salvar checkpoints em:

```
/raid/$USER/checkpoints/
```

- O intervalo de checkpoint deve considerar:
 - tempo total do job
 - custo de recomputação
 - tamanho do estado salvo

Exemplos comuns

- **Treinamento de redes neurais**: salvar modelo, otimizador e epoch atual.
- **Simulações longas**: salvar estado intermediário.
- **Pipelines multi-etapas**: persistir saída de cada fase.

Integração com Slurm (boas práticas)

- Use jobs **reentrantes**, capazes de retomar do último checkpoint.
- Considere usar:

```
#SBATCH --signal=B:SIGUSR1@300
```

para capturar sinais antes do término forçado.

- Trate o sinal no código para salvar checkpoint antes da finalização.

⚠ Jobs que não implementam checkpoint assumem o risco de perda total em manutenções.

⚠ Política de Uso de Recursos

- GPUs são recursos compartilhados.
- Prioridade pode ser aplicada conforme política institucional.
- Jobs devem prever checkpoint sempre que possível.
- Uso indevido pode resultar em bloqueio automático.



Resumo

- Armazene dados em `/raid/$USER`
- Nunca execute GPU fora do Slurm
- Use partição correta
- Monitore seus jobs
- Utilize containers quando necessário
- Use o Discord para suporte